

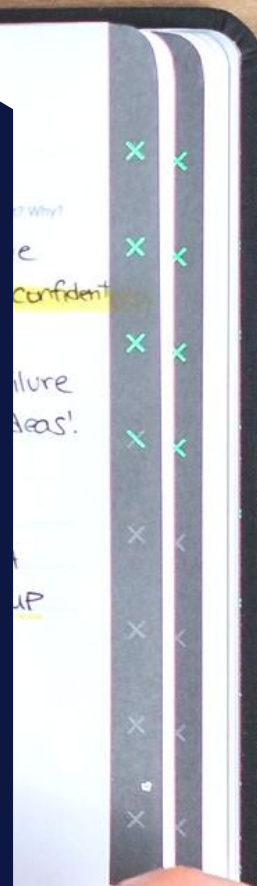


Práctica 02

Patrones de Diseño de
APIs

Elaborado por:

Manuel William Villa Quishpe



Actividades

Práctica: Uso de POSTMAN para probar una API RESTful (PayPal Subscriptions API)

Estudiante: Manuel William Villa Quishpe

Materia: Patrones de Diseño de APIs

Docente: Ing. Patsy Malena Prieto, MSc.

Programa: Maestría en Software, Universidad Politécnica Salesiana

Descripción de la actividad

La presente práctica tuvo como objetivo identificar las ventajas y desventajas del uso de una API RESTful mediante su consumo directo desde un cliente HTTP. Para ello se examinó el funcionamiento de Postman, organizando el trabajo en un *workspace* de la asignatura y en una colección con las peticiones de la API de Suscripciones de PayPal en su entorno Sandbox. Se revisó la sintaxis básica de los métodos HTTP GET, POST, PUT y DELETE, junto con sus códigos de estado, cabeceras y cuerpos en formato JSON. Finalmente, se realizaron ejercicios que simulan la interacción entre el frontend y el backend: se obtuvo un token OAuth 2.0 con un script automático, se listaron los planes de facturación, se creó un plan con ciclos TRIAL y REGULAR, se actualizó su umbral de pagos fallidos con PATCH (equivalente a PUT en PayPal) y se desactivó el plan mediante POST (equivalente a DELETE).

Captura de Pantallas Funcionales

1. Autenticación OAuth 2.0 (POST Generate access_token)

Petición POST /v1/oauth2/token con Basic Auth (client_id y client_secret). La respuesta 200 OK entrega el access_token Bearer, que el script «After response» guarda automáticamente como variable de la colección.

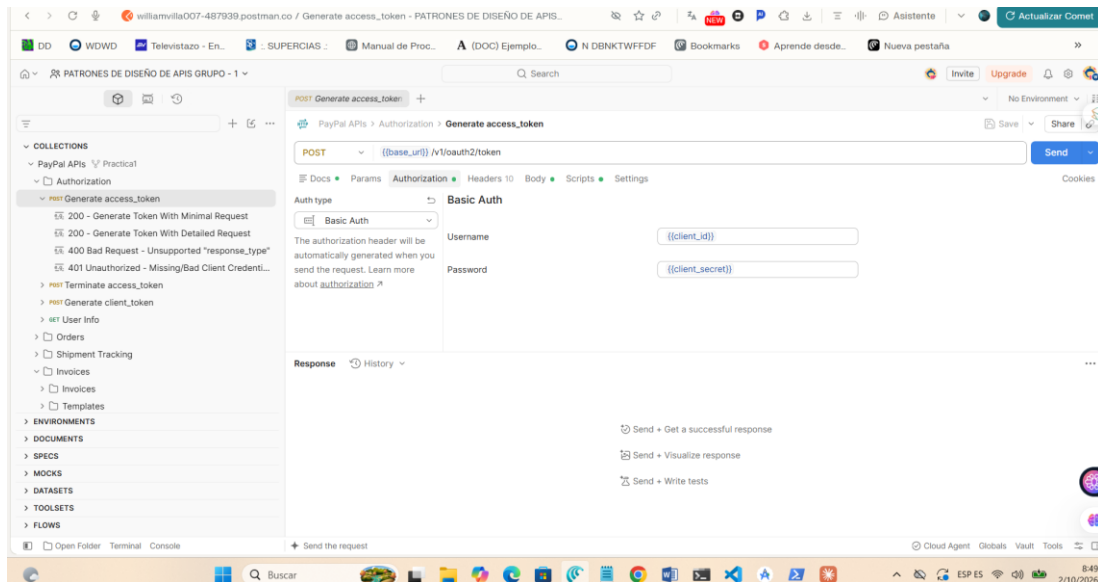


Figura 1. Configuración de Basic Auth con las credenciales de la aplicación Sandbox.

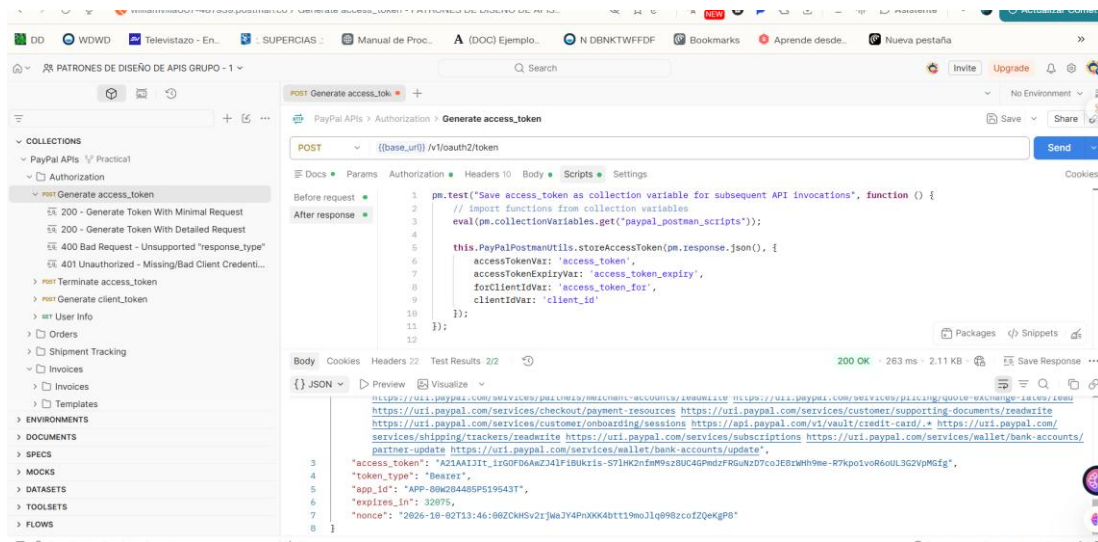


Figura 2. Script automático de almacenamiento del token y respuesta 200 OK con el access_token.

2. GET List plans

Petición GET `/v1/billing/plans` con `page_size=10`, `page=1` y `total_required=true`. La respuesta **200 OK** devuelve el arreglo `plans` paginado: una operación de solo lectura, segura e idempotente.

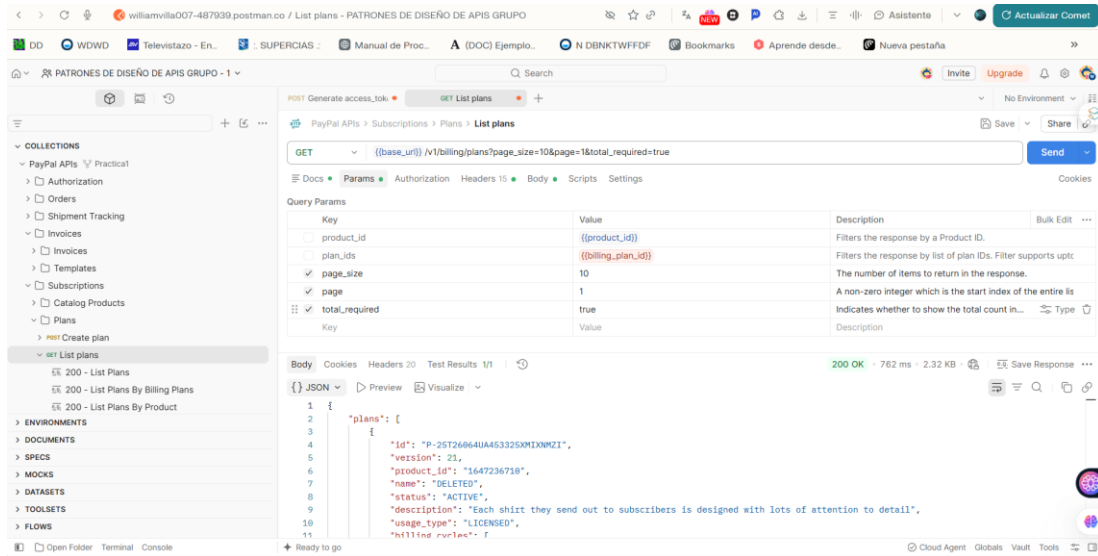


Figura 3. Listado de planes de facturación con respuesta 200 OK.

3. POST Create plan

Petición POST `/v1/billing/plans` con cuerpo JSON que define un ciclo TRIAL (1 USD) y uno REGULAR (44 USD, 12 meses). La respuesta **201 Created** devuelve el plan creado con el id `P-8J748619R2551061NNK73UAQ`.

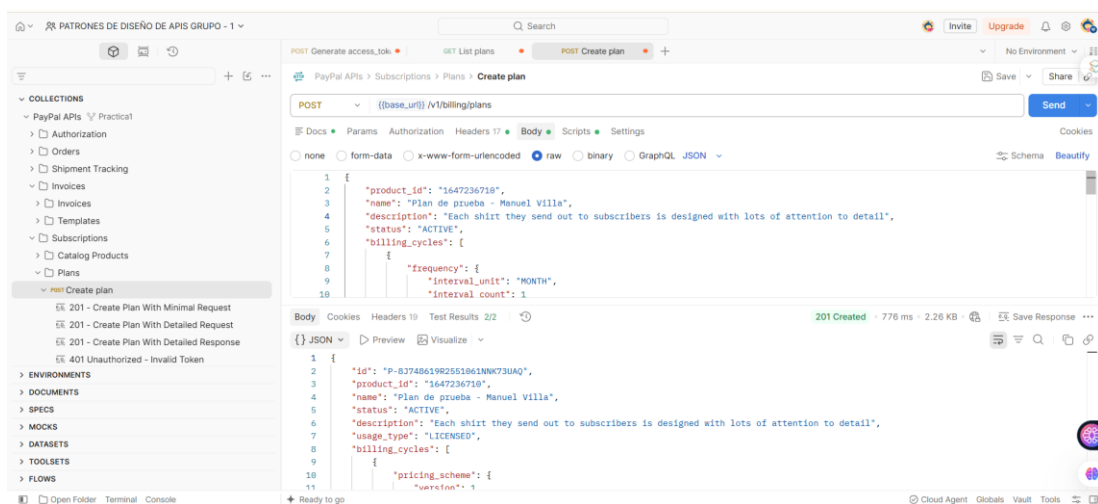


Figura 4. Creación del plan con ciclos TRIAL y REGULAR y respuesta 201 Created.

4. PATCH Update plan (equivalente a PUT)

Petición PATCH `/v1/billing/plans/:billing_plan_id` con un JSON Patch (op: replace) que cambia `payment_failure_threshold` de 3 a 7. PayPal no expone PUT para planes; responde **204 No Content**, éxito sin cuerpo.

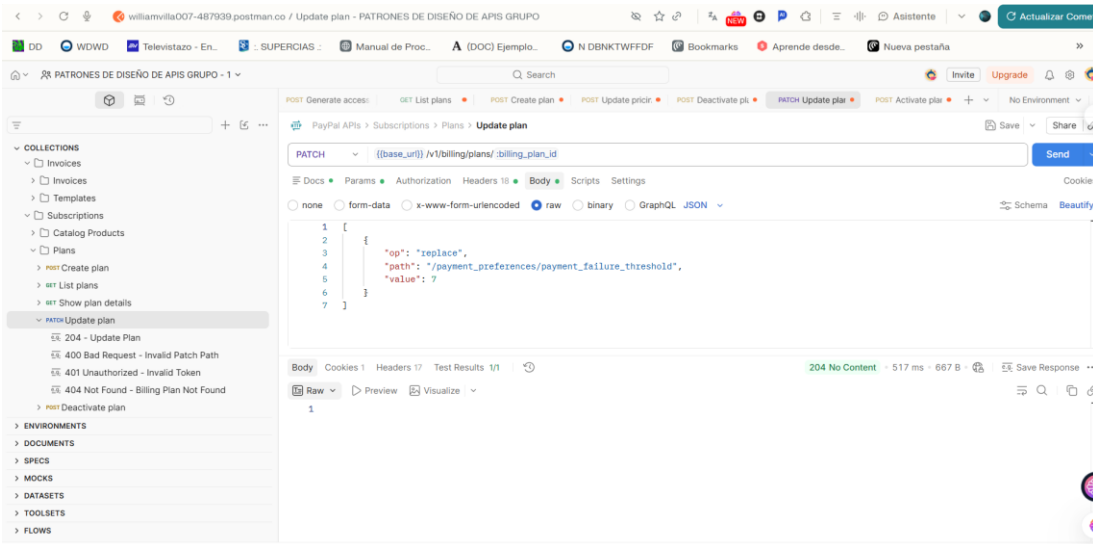


Figura 5. Actualización del umbral de pagos fallidos con respuesta 204 No Content.

5. POST Deactivate plan (equivalente a DELETE)

Petición POST `/v1/billing/plans/:billing_plan_id/deactivate`, sin cuerpo. PayPal no elimina planes físicamente: aplica un borrado lógico que los deja en estado INACTIVE y responde **204 No Content**.

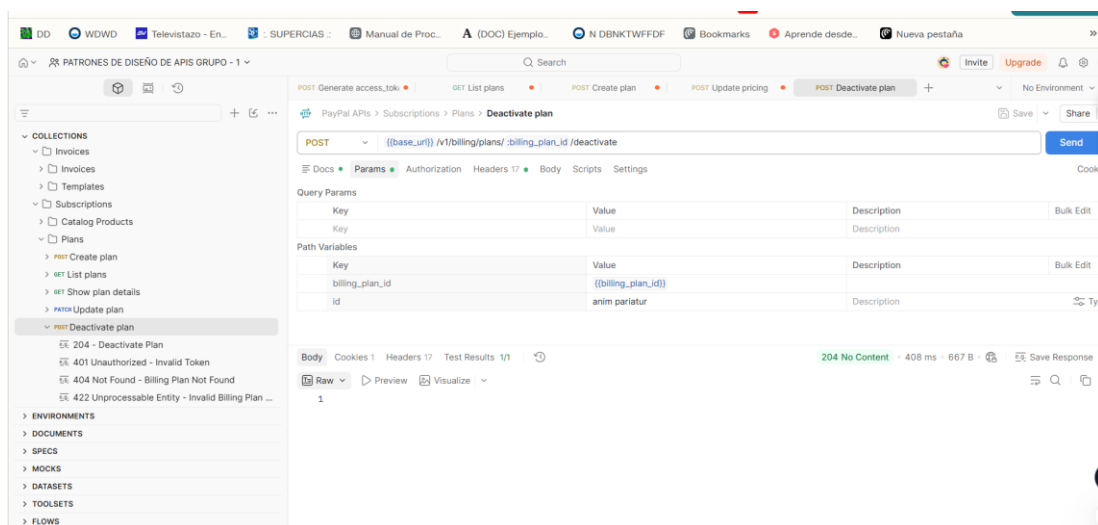


Figura 6. Desactivación del plan con respuesta 204 No Content.

Código Fuente / Archivos JSON

Como respaldo de la práctica se incluyen los archivos JSON utilizados en la colección de Postman, copiados tal como se emplearon en las peticiones.

create_plan.json

Cuerpo de la petición POST /v1/billing/plans (Create plan).

```
{
  "product_id": "1647236710",
  "name": "Plan de prueba - Manuel Villa",
  "description": "Each shirt they send out to subscribers is designed with lots of attention to detail",
  "status": "ACTIVE",
  "billing_cycles": [
    {
      "frequency": {
        "interval_unit": "MONTH",
        "interval_count": 1
      },
      "tenure_type": "TRIAL",
      "sequence": 1,
      "total_cycles": 1,
      "pricing_scheme": {
        "fixed_price": {
          "value": "1",
          "currency_code": "USD"
        }
      }
    },
    {
      "frequency": {
        "interval_unit": "MONTH",
        "interval_count": 1
      },
      "tenure_type": "REGULAR",
      "sequence": 2,
      "total_cycles": 12,
      "pricing_scheme": {
        "fixed_price": {
          "value": "44",
          "currency_code": "USD"
        }
      }
    }
  ],
  "payment_preferences": {
    "auto_bill_outstanding": true,
    "setup_fee": {
      "value": "10",
      "currency_code": "USD"
    },
    "setup_fee_failure_action": "CONTINUE",
    "payment_failure_threshold": 3
  },
  "taxes": {
    "percentage": "10",
    "inclusive": false
  }
}
```

patch_plan.json

Cuerpo de la petición PATCH /v1/billing/plans/{id} (Update plan), en formato JSON Patch.

```
[
  {
    "op": "replace",
    "path": "/payment_preferences/payment_failure_threshold",
    "value": 7
  }
]
```

update_pricing.json

Cuerpo de la petición POST /v1/billing/plans/{id}/update-pricing-schemes (Update pricing), que fija en 50 USD el precio del ciclo REGULAR (secuencia 2).

```
{
  "pricing_schemes": [
    {
      "billing_cycle_sequence": 2,
      "pricing_scheme": {
        "fixed_price": {
          "value": "50",
          "currency_code": "USD"
        },
        "roll_out_strategy": {
          "effective_time": "2022-11-01T00:00:00Z",
          "process_change_from": "NEXT_PAYMENT"
        }
      }
    }
  ]
}
```

get_response.json

Respuesta de la petición GET /v1/billing/plans (List plans).

```
{
  "plans": [
    {
      "id": "P-25T26064UA453325XMIXNMZI",
      "version": 21,
      "product_id": "1647236710",
      "name": "DELETED",
      "status": "ACTIVE",
      "description": "Each shirt they send out to subscribers is designed with lots of attention to detail",
      "usage_type": "LICENSED",
      "billing_cycles": [
        {
          "pricing_scheme": {
```



```

        "version": 20,
        "fixed_price": {
            "value": "44",
            "currency_code": "USD"
        }
    }
}
],
"links": [
    {
        "href": "https://api.sandbox.paypal.com/v1/billing/plans?page_size=10&page=1",
        "rel": "self",
        "method": "GET",
        "encType": "application/json"
    },
    {
        "href": "https://api.sandbox.paypal.com/v1/billing/plans?page_size=10&page=2",
        "rel": "next",
        "method": "GET",
        "encType": "application/json"
    },
    {
        "href": "https://api.sandbox.paypal.com/v1/billing/plans?page_size=10&page=6553",
        "rel": "last",
        "method": "GET",
        "encType": "application/json"
    }
]
}

```

Conclusiones

1. La API de Suscripciones de PayPal confirma la principal ventaja del estilo REST: una interfaz uniforme y sin estado, en la que cada recurso se identifica por su URI y el resultado se comunica con códigos HTTP estándar (200, 201 y 204), lo que permite a cualquier cliente consumirla sin conocer su implementación. Su desventaja es que la correspondencia entre operaciones CRUD y verbos HTTP no es estricta: la actualización se resuelve con PATCH y JSON Patch en lugar de PUT, y la eliminación con una acción POST sobre el subrecurso **/deactivate** en lugar de DELETE, por lo que el consumidor depende de la documentación de cada proveedor.

2. Postman permite simular el papel del frontend frente al backend sin escribir un cliente propio: las variables de colección (`{{base_url}}`, `{{client_id}}`) y los scripts posteriores a la respuesta automatizan el flujo OAuth 2.0 y encadenan las peticiones, mientras que los *tests* validan cada respuesta. Como limitación, al ser REST un estilo sin estado, el token Bearer debe enviarse en todas las peticiones y renovarse cuando expira, y las pruebas manuales no sustituyen a una suite automatizada integrada al ciclo de desarrollo.

Bibliografía

- Bryan, P., & Nottingham, M. (Eds.). (2013). *JavaScript Object Notation (JSON) Patch* (RFC 6902). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc6902>
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc9110>
- Hardt, D. (Ed.). (2012). *The OAuth 2.0 authorization framework* (RFC 6749). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc6749>
- Mozilla. (s. f.). *Métodos de petición HTTP*. MDN Web Docs. Recuperado el 2 de octubre de 2026, de <https://developer.mozilla.org/es/docs/Web/HTTP/Methods>
- PayPal. (s. f.). *Subscriptions v1*. PayPal Developer. Recuperado el 2 de octubre de 2026, de <https://developer.paypal.com/docs/api/subscriptions/v1/>
- Postman. (s. f.). *Postman Learning Center*. Recuperado el 2 de octubre de 2026, de <https://learning.postman.com/docs/>
- Prieto, P. M. (s. f.). *Guía de práctica de laboratorio N.º 02: Uso de POSTMAN* [Material de clase]. Maestría en Software, Universidad Politécnica Salesiana.

Patrones de Diseño de APIs

The screenshot shows a GitHub repository interface for 'Patrones-Diseño-APIs-Practica1'. The repository is public and has 2 commits. The commit history table is as follows:

Commit	Author	Message	Time
cb42701	ManuelVilla00727	Create README.md	1 minute ago
		create_plan.json	2 minutes ago
		get_response.json	2 minutes ago
		patch_plan.json	2 minutes ago
		update_pricing.json	2 minutes ago

The README file is titled 'Práctica 1 - Uso de POSTMAN (PayPal Subscriptions API)'. The content of the README is:

Estudiante: Manuel William Villa Quiroga. Materia: Patrones de Diseño de APIs. Docente: Ing. Patricio Malana Prieto. MSc.

The right sidebar shows the repository's metadata: 'Práctica 1 - Uso de Postman con PayPal API', 0 stars, 0 watching, 0 forks, and 1 contributor.